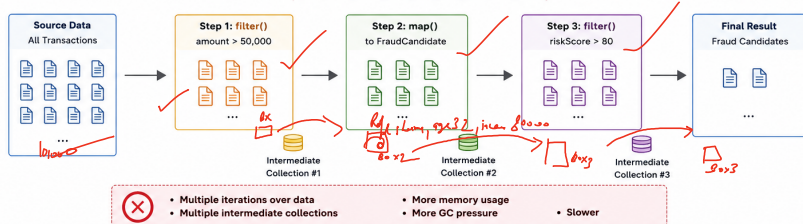


Session 1 Topics:

WITHOUT STREAM FUSION

(Creates Intermediate Collections)

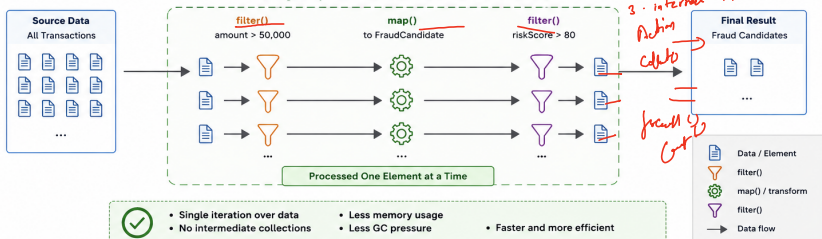


```
List<Transaction> step1  
List<FraudCandidate> step2  
List<FraudCandidate> finalList
```

```
transactions.stream()  
    .filter(...)  
    .map(...)  
    .filter(...)  
    .collect(...)
```

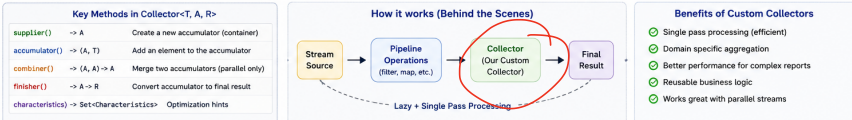
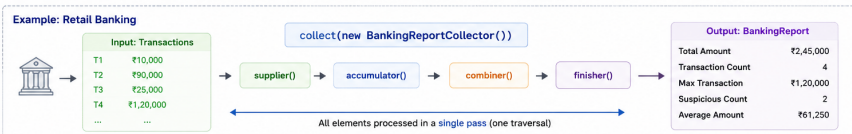
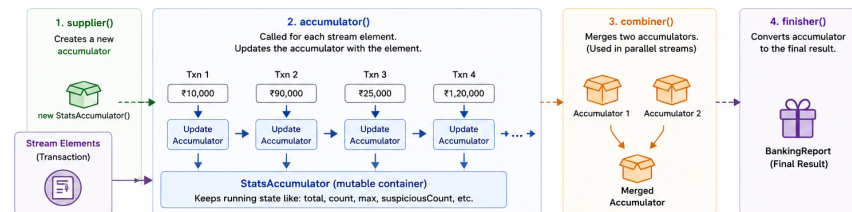
WITH STREAM FUSION

(Single Pipeline - No Intermediate Collections)



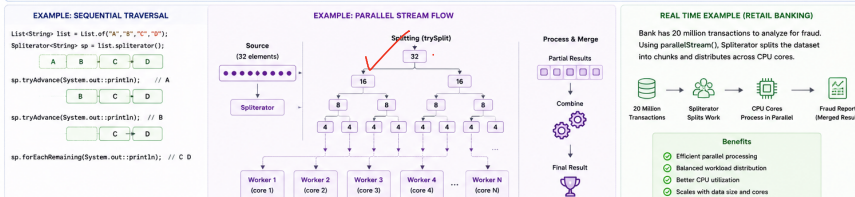
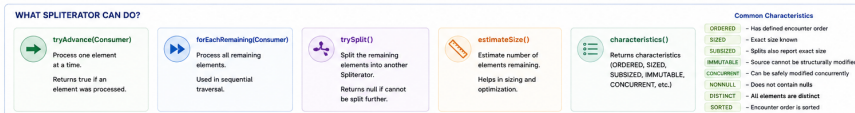
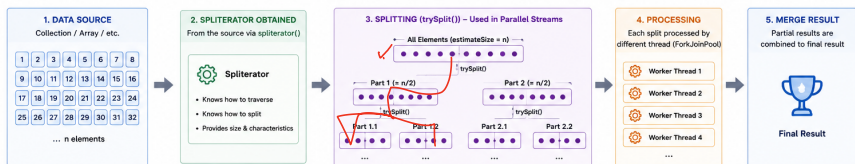
Custom Collectors in Java (How it Works)

A Custom Collector defines HOW to collect stream elements into a final result.
It tells the Stream API: how to create a container, how to add elements, how to combine, and how to finish.



Splitter Mechanics in Java

The engine behind Streams that enables efficient traversal and parallel processing



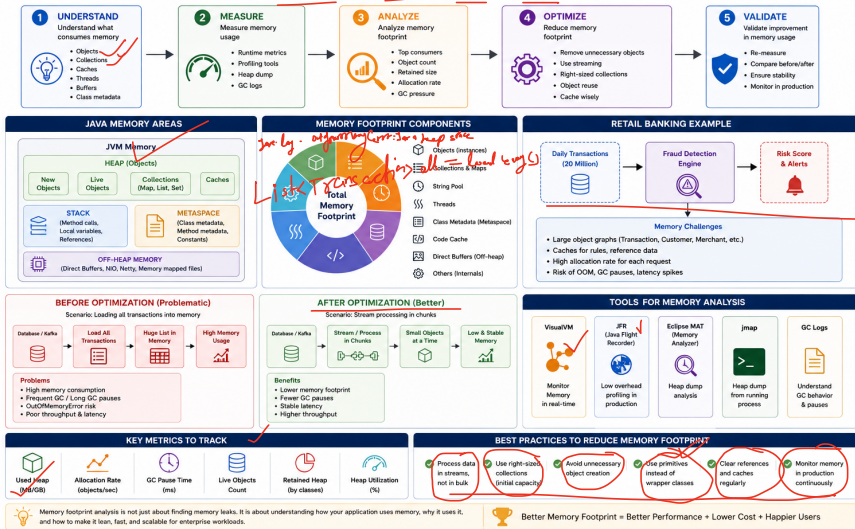
Key Takeaway Splitter is the unsung hero of Java Streams. It enables lazy traversal, efficient splitting, and provides hints about the data so that the Stream framework can execute operations in the most optimized way.

SR
2. Total Transaction
2. Count
3. Highest Transaction
4. Suspicious Transaction
(50,000)

$u_1 \rightarrow 1000$
 $u_2 \rightarrow 1000$
 $A \rightarrow B$
 $B \rightarrow 1000$
 $B \rightarrow 1000$
 $B \rightarrow 1000$
Splitter
= split + Iterator
java.util.Spliterators

MEMORY FOOTPRINT ANALYSIS IN JAVA

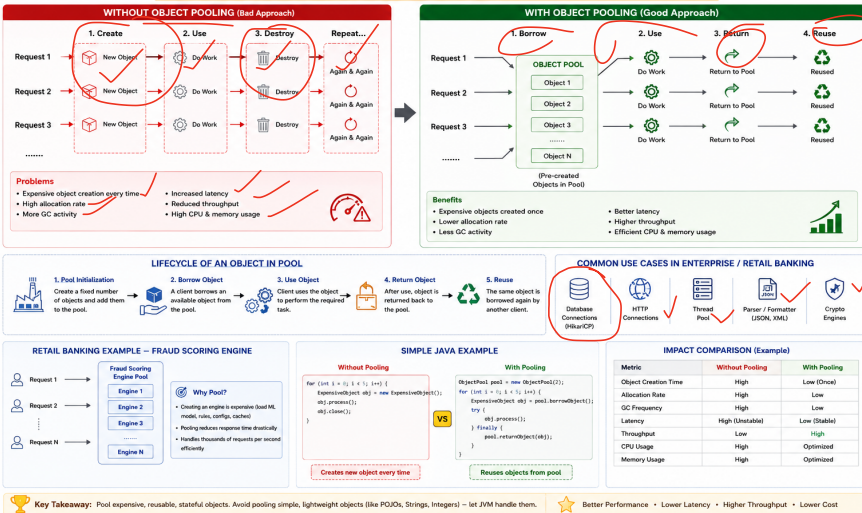
Understand - Measure - Analyze - Optimize - Validate



OBJECT POOLING IN JAVA - REUSE INSTEAD OF RECREATE

Borrow -> Use -> Return -> Reuse

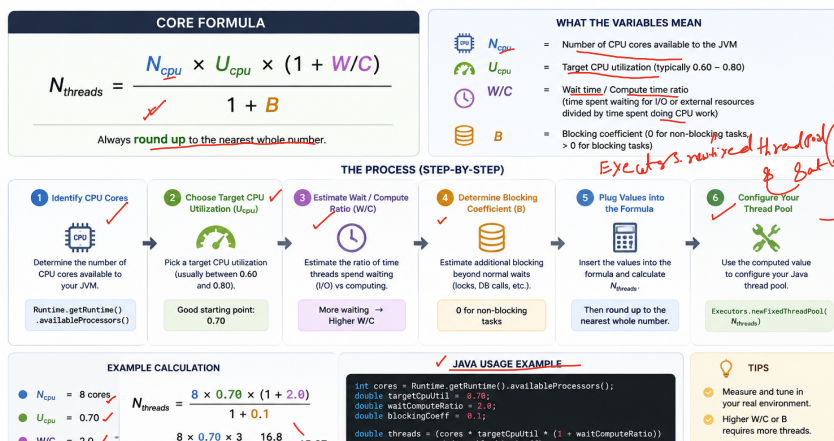
Object Pooling is a design pattern that reuses expensive objects instead of creating new ones repeatedly. It improves performance, reduces GC pressure, and increases throughput.



Session 2 Topics:

Thread Pool Sizing Math in Java

Right-size your thread pool for optimal performance



$$W/C = \frac{\text{time spent waiting}}{\text{time spent doing work}}$$

$$\text{Threads} = \text{CPU Cores} \times (1 + W/C)$$

$$B = \frac{\text{Wait time}}{\text{Wait time} + \text{Compute time}}$$

$$W = 80 + 120 = 2000$$

$$\text{Cores} = 5000$$

$$R = \frac{2000}{5000 + 50} = \frac{2000}{5050} \approx 0.4$$

$$= 0.8$$

⇒) Backpressure Modeling (⇐)

 **Lock-Free Collections**
High-performance concurrent data structures without locks

